Reinforcement Learning on Acrobot-v1:

A Comparative Study of Tabular and Function Approximation Methods

April 21, 2026

Contents

1 Introduction **2**

2 Environment **2**

3 State Representation and Discretisation **2**
3.1 Coordinate Transform . . . 2
3.2 Discretisation Scheme . . . 3

4 Algorithms **3**
4.1 SARSA . . . 3
4.2 Q-Learning . . . 4
4.3 n-Step SARSA . . . 4
4.4 SARSA(λ) - True Online Variant . . . 4
4.5 REINFORCE with Baseline (Tabular) . . . 4

5 Hyperparameter Optimisation **5**
5.1 Optimisation Framework . . . 5
5.2 Hyperparameter Search Spaces . . . 5
5.3 Epsilon Decay Reparametrisation . . . 6
5.4 Best Hyperparameters and Importance . . . 6

6 Experimental Results **7**
6.1 Training Protocol . . . 7
6.2 Learning Curves . . . 7
6.3 Evaluation Summary . . . 8
6.4 Policy Coverage . . . 9

7 Discussion **9**
7.1 Why SARSA Outperformed Q-Learning . . . 9
7.2 Implications for Credit Assignment . . . 10
7.3 REINFORCE . . . 10

8 Conclusion **10**

1 Introduction

Reinforcement Learning is a branch of Machine Learning in which an agent follows a learned policy to interact with the environment, with the goal of optimising the expected cumulative rewards. Specifically, this report focuses on exploring a classic control problem that can be formulated as a reinforcement learning problem. This exploration consists of a variety of algorithms, trained and tested, to compare the agents' performance in the Acrobot-v1 environment as formulated from Gymnasium [1]. One axis in which the breadth of algorithms arises is the Temporal Difference and Monte Carlo spectrum. With algorithms like SARSA, Q-Learning, and n-step SARSA, it is possible to study how Temporal Difference plays a role in policy learning for a double-pendulum. The REINFORCE algorithm [2] serves the purpose of understanding the other end of the spectrum, and SARSA(λ) provides an insight into the eligibility trace that bridges the two ends of that range.

Furthermore, an extension to the project is provided by exploring a variant of the REINFORCE implementation with artificial neural networks, as a way to address a significant drawback that the tabular implementation faces. While the rest of the algorithms handle a tabular representation of the control problem, REINFORCE suffers from sparse policy gradient updates that arise when discretising a continuous state space. With these objectives in mind, the report is structured as follows. Section 2 describes the problem and environment. Section 3 details the state representation and discretisation scheme. Section 4 presents the pseudocode and implementation decisions for each algorithm. Section 5 describes the hyper-parameter optimisation pipeline. Section 6 presents the experimental results. Section 7 analyses the findings, and section 8 summarises the key contributions and limitations.

2 Environment

The environment in which the agents interact is a classic control problem of swinging a double pendulum, with the goal of raising the end of the second link above a threshold. Gymnasium provides six dimensions of continuous observations. These consist of the cosine and sine of the angles formed by the two pendulum joints, plus their respective angular velocities. Trigonometric functions range between -1 and 1, and angular velocities are capped to a magnitude of 4π for the first joint and 9π for the second. The action space is discrete, corresponding to three actions: torque to the left, no torque, and torque to the right, respectively. The reward at any given time step is -1 until success, and an episode terminates after at most 500 steps. Thus, the problem asks for the maximisation of cumulative rewards, implying a minimisation of the number of steps required to reach the threshold. A higher (less negative) cumulative reward therefore corresponds to a faster solution.

3 State Representation and Discretisation

3.1 Coordinate Transform

The raw observation uses cosine-sine pairs to encode each joint angle, which introduces a redundancy: both $(\cos\theta, \sin\theta)$ and any rotation thereof satisfy $\cos^2\theta + \sin^2\theta = 1$, meaning

the 6D observation actually encodes only 4 degrees of freedom. To remove this redundancy and obtain a more compact, interpretable state, the arctan2 function is applied to recover the original angles:

$$\theta_i = \arctan2(\sin\theta_i,\ \cos\theta_i), \quad i \in \{1, 2\}. \tag{1}$$

The resulting 4D state $[\theta_1, \theta_2, \omega_1, \omega_2]$ is both more compact and physically interpretable, and is used as the basis for all subsequent representations.

3.2 Discretisation Scheme

For tabular algorithms, the continuous 4D state must be mapped to a finite set of indices. Bin boundaries are defined as:

$$\mathbf{l} = [-\pi,\ -\pi,\ -6,\ -10], \qquad \mathbf{h} = [\ \pi,\ \ \pi,\ \ 6,\ \ 10], \tag{2}$$

with bin counts $\mathbf{b} = [10,\ 10,\ 12,\ 12]$ for angles and velocities respectively, yielding $10 \times 10 \times 12 \times 12 = 14{,}400$ discrete states. The discretisation maps each continuous coordinate to an integer bin index via:

$$d_i = \left\lfloor \mathrm{clip}\left(\frac{s_i - l_i}{h_i - l_i},\ 0,\ 1\right) \times (b_i - 1) \right\rfloor. \tag{3}$$

The bin choice was explored empirically: the finer grids increase state coverage without a proportional improvement in performance; coarser grids cause excessive aliasing and policy learning becomes difficult. The scheme $[10, 10, 12, 12]$ was selected as a balance between state space resolution and the feasibility of adequate coverage during training. All tabular algorithms use the same discretisation to ensure a fair comparison.

4 Algorithms

For all algorithms, except REINFORCE, maintain a Q-table implemented as a `defaultdict` initialised to zeros for unseen states, and use an ε-greedy policy throughout training. The following parameters are set as global constants for the applicable algorithms, with $\gamma = 0.99$ and ε decays exponentially from 1.0 toward $\varepsilon_{\min} = 0.01$. The setting $\varepsilon_{\min}$ of 0.01 arises from a hypothesis: deterministic policies are preferable because these are predictable, tend to be precise and consistent over stochastic policies. Such attributes could be beneficial on real-world control systems.

4.1 SARSA

SARSA [2] is an on-policy TD(0) algorithm. At each step, the agent takes action A_t under the current ε-greedy policy, observes R_{t+1} and S_{t+1}, selects A_{t+1} under the same policy, and updates:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]. \tag{4}$$

The on-policy bootstrap target $Q(S_{t+1}, A_{t+1})$ reflects the action the agent would actually take, making SARSA conservative relative to Q-Learning. SARSA is used here as the baseline, trained with arbitrary default parameters ($\alpha = 0.15$, ε-decay $= 0.985$), without hyper-parameter optimisation to contextualise the results of the other algorithms.

4.2 Q-Learning

Q-Learning [2] is an off-policy TD(0) algorithm. The update replaces the on-policy bootstrap target from SARSA with the greedy maximum:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big]. \tag{5}$$

By learning the value of the greedy policy regardless of the behaviour policy used to collect data, Q-Learning can in principle converge to the optimal Q-function. However, under discretisation aliasing, the max operator can overestimate the value of actions that are only good for a subset of the aliased states. SARSA's on-policy target is more conservative and, as shown later in section 6, performs better in this setting.

4.3 n-Step SARSA

n-Step SARSA [2] generalises TD(0) by accumulating n steps of actual rewards before bootstrapping from the Q-table:

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n}). \tag{6}$$

The Q-table update then uses $G_{t:t+n}$ in place of the one-step TD target. The algorithm bridges the TD ($n = 1$) and Monte Carlo ($n \to \infty$) extremes. Later on, it can be seen that the hyper-parameter optimisation pipeline selected $n = 6$. This implies capturing medium-range dependencies consistent with the physical structure of the Acrobot swing-up manoeuvre, which requires coordinated actions.

4.4 SARSA(λ) - True Online Variant

The True Online SARSA(λ) algorithm [2] uses eligibility traces to propagate credit through the episode history. In the tabular setting, one common implementation is through linear function approximation with one-hot features $\mathbf{x}(s, a)$, where $Q(s, a) \approx \mathbf{w}^\top \mathbf{x}(s, a)$. The dutch trace update is:

$$\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \big(1 - \alpha\gamma\lambda\, \mathbf{z}^\top \mathbf{x}\big)\mathbf{x}, \tag{7}$$

and the weight update includes a correction term that distinguishes True Online from standard SARSA(λ):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(\delta + Q - Q_{\text{old}})\mathbf{z} - \alpha(Q - Q_{\text{old}})\mathbf{x}, \tag{8}$$

where $\delta = R + \gamma Q' - Q$ is the TD error and Q_{old} is the value of the current state-action pair from the previous step. This correction makes the algorithm equivalent to the online forward-view TD(λ), whereas standard SARSA(λ) only approximates it.

4.5 REINFORCE with Baseline (Tabular)

REINFORCE [2] is a Monte Carlo policy gradient algorithm. The policy is represented as a softmax over preference logits $h(s, a)$:

$$\pi(a \mid s) = \frac{\exp(h(s, a))}{\sum_b \exp(h(s, b))}. \tag{9}$$

The REINFORCE with baseline variant [2] augments the update with a learned state-value function $\hat{v}(s)$ to reduce variance. For each step t of a generated episode, the updates are:

$$\delta_t = G_t - \hat{v}(S_t), \tag{10}$$

$$\hat{v}(S_t) \leftarrow \hat{v}(S_t) + \alpha^w \delta_t, \tag{11}$$

$$h(S_t, A_t) \leftarrow h(S_t, A_t) + \alpha^\theta \gamma^t \delta_t \nabla \ln \pi(A_t \mid S_t), \tag{12}$$

where $G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ is the discounted return from step t. The γ^t factor is included as specified in the pseudocode of Sutton and Barto [2]. Raw (non-normalised) returns are used throughout so that the baseline $\hat{v}$ is trained on a consistent scale. The tabular implementation stores logits h and values $\hat{v}$ as `defaultdict` structures, updating only visited states.

5 Hyperparameter Optimisation

5.1 Optimisation Framework

Hyperparameter optimisation for all algorithms (except the SARSA baseline) was performed using Optuna [3] with a SQLite storage backend. The objective function evaluates each hyperparameter configuration across 7 RNG seeds and returns the Interquartile Mean (IQM) of the per-seed scores [4], where the score for each seed is the mean reward over the final 10% of training episodes. The IQM trims the top and bottom 25% of seed scores before averaging, making the objective resistant to outlier seeds that either unusually succeed or fail due to random initialisation. A MedianPruner is applied at each intermediate seed step, terminating unpromising trials early based on comparison to the median of completed trials. Each study runs for 64 trials.

The pipeline was designed to be resumable: on each call, it checks the persistent database for existing completed and pruned trials, removes any trials left in incomplete states (e.g. due to interrupted runs), and continues optimisation only for the remaining trials needed to reach the target of 64. This allowed optimisation to be distributed across multiple sessions without redundancy.

5.2 Hyperparameter Search Spaces

Table 1 summarises the search spaces used for each algorithm. The learning rate α is searched on a log scale given its multi-order-of-magnitude sensitivity. The `floor_fraction` parameter is specific to this work and is described in section 5.3. All other parameters are fixed across algorithms: $\gamma = 0.99$, $\varepsilon = 1.0$, $\varepsilon_{\min} = 0.01$, $n_{\text{episodes}} = 300$ for TD methods and 600 for REINFORCE.

Table 1: Hyperparameter search spaces used in Optuna optimisation.

Algorithm	Parameter	Range	Scale
All TD methods	α	$[10^{-3},\ 0.5]$	log
All TD methods	`floor_fraction`	$[0.3,\ 0.8]$	linear
n-step SARSA	n	$[2,\ 14]$	integer
SARSA(λ)	λ	$[0.0,\ 1.0]$	linear
REINFORCE	α_θ	$[10^{-4},\ 10^{-2}]$	log
	α_w	$[10^{-4},\ 0.3]$	log

5.3 Epsilon Decay Reparametrisation

A key methodological challenge arose from the mismatch between the HPO episode budget and the final training budget. If ε-decay is optimised directly as a scalar, a value that produces a sensible exploration schedule at 300 episodes will cause premature convergence when applied naively to 5000 episodes. For example, a decay of 0.960 reaches $\varepsilon_{\min} = 0.01$ at episode ≈ 114 out of 300 during HPO (38% of total episodes). In contrast, at episode ≈ 114 out of 5000 during final training (2.28% of total episodes), leaves the agent in near-greedy mode for the remaining 94% of training with an uninitialised Q-table.

To address this, ε-decay is not optimised directly. Instead, Optuna suggests a `floor_fraction` $f \in [0.3, 0.8]$, representing the fraction of the training budget over which exploration occurs. The decay is then computed deterministically:

$$\varepsilon_{\text{decay}} = \varepsilon_{\min}^{1/(f \cdot N)}, \tag{13}$$

where N is the episode budget. When the same f is applied at the final training budget $N = 5000$, the shape of the exploration schedule is preserved regardless of the number of episodes. This reparametrisation was applied consistently across all TD algorithms.

The importance of this fix was confirmed empirically: an early run of SARSA(λ) with `floor_fraction` $= 0.311$ produced a decay computed for the HPO budget of 300 episodes. When applied without correction to 2000 final episodes, ε reached the floor at episode 93, resulting in 0% greedy success rate despite 100% stochastic success rate. Retraining with the corrected decay recovered full performance.

5.4 Best Hyperparameters and Importance

Table 2 reports the best trial parameters found by Optuna for each algorithm.

Table 2: Best hyperparameters found by Optuna (64 trials, 7 seeds, IQM objective).

Algorithm	Parameter	Value	HPO Score (IQM)
Q-Learning	α	0.3110	-288.7
	`floor_fraction`	0.4178	
n-step SARSA	α	0.4664	-250.7
	`floor_fraction`	0.6347	
	n	6	
SARSA(λ)	α	0.4114	-165.8
	`floor_fraction`	0.3112	
	λ	0.9391	
REINFORCE (tabular)	α^θ	3.6×10^{-3}	-497.5
	α^w	6.3×10^{-3}	

Across TD methods, α was consistently identified as the most important hyperparameter, with `floor_fraction` secondary. This is expected, because the learning rate directly controls the magnitude of Q-value updates at every step, while exploration duration has a more indirect effect on final performance. The optimal $n = 6$ for n-step SARSA indicates a preference for medium-range credit assignment over either pure TD or near-MC returns, which is consistent with the physical structure of the swing-up task discussed in section 7. The $\lambda = 0.939$ for SARSA(λ) indicates a strong preference for long credit assignment horizons, with an effective trace decay of $0.939^{50} \approx 0.05$ at roughly 50 steps.

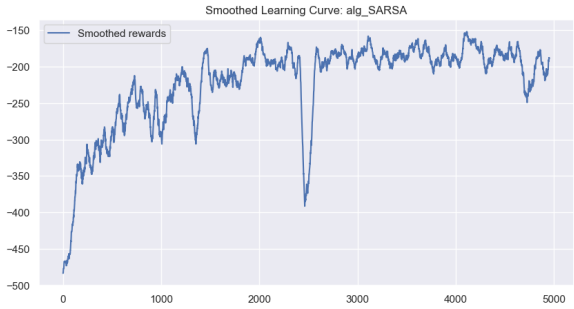
6 Experimental Results

6.1 Training Protocol

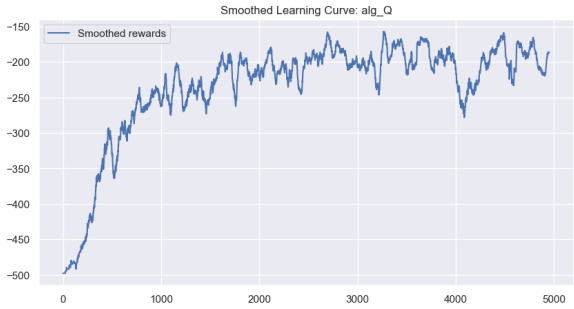
All final models were trained for 5000 episodes with seed 42. The ε-decay for TD methods was recomputed from the Optuna `floor_fraction` using $N = 5000$ as described in section 5.3. Evaluation used 20 episodes with greedy action selection (argmax over Q-values or policy logits), with per-episode seeds drawn as 42 + episode index to ensure reproducibility and diversity. The evaluation metrics reported are mean cumulative reward, standard deviation, mean episode length, and success rate (fraction of episodes solved in fewer than 500 steps). All trained policies were saved to disk: tabular methods as `defaultdict`-reconstructed pickle file. These same metrics were also calculated for the same policy being treated as a stochastic policy (through softmax) as additional information.

6.2 Learning Curves

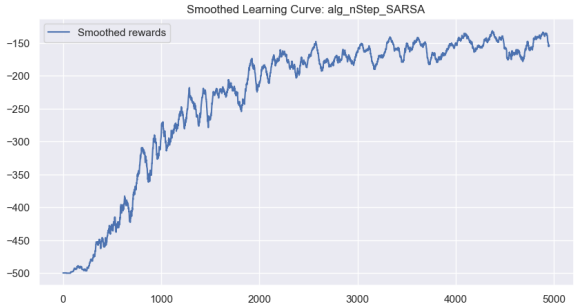
Figure 1 shows the smoothed learning curves (50-episode moving average) for each algorithm over 5000 training episodes.



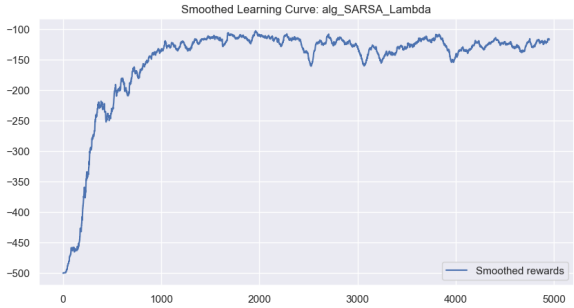
(a) SARSA (baseline)



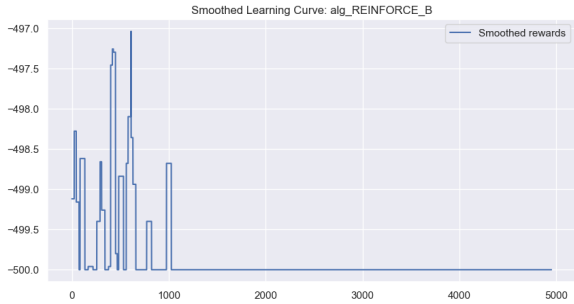
(b) Q-Learning



(c) n-step SARSA ($n = 6$)



(d) SARSA(λ), $\lambda = 0.939$



(e) REINFORCE with baseline (tabular)

Figure 1: Smoothed learning curves (50-episode window) for all algorithms over 5000 training episodes. Higher (less negative) reward indicates faster episode completion.

All TD methods show a characteristic initial plateau followed by rapid improvement as the Q-table accumulates sufficient coverage, and then stabilisation with residual variance from the stochastic policy. n-step SARSA and SARSA(λ) converge to higher reward levels and with lower variance at convergence than the single-step methods. The tabular REINFORCE curve remains near -500 throughout, confirming the failure of the tabular policy gradient in this setting.

6.3 Evaluation Summary

Table 3 summarises the greedy evaluation results for all algorithms.

Table 3: Greedy evaluation results (20 episodes). Higher mean reward and lower standard deviation indicate better performance.

Algorithm	Mean Reward	Std	Mean Steps	Success Rate
SARSA (baseline)	-156.4	43.39	157.4	1.00
Q-Learning	-182.95	31.51	183.95	1.00
n-step SARSA	-135.2	35.38	136.2	1.00
SARSA(λ)	-126.05	29.21	127.05	1.00
REINFORCE (tabular)	-500	0.0	500	0.00

SARSA(λ) achieves the best mean reward of -126.05 with the lowest standard deviation among the multi-step methods, indicating both the fastest average solution and the most consistent policy. n-step SARSA achieves the second-best mean reward but with higher variance ($\sigma = 35.38$), suggesting the learned policy is strong on average but less robust. SARSA and Q-Learning perform as expected: SARSA obtains a better mean reward but has high variance. Q-learning shows more consistent results, but a worse mean reward. The tabular REINFORCE fails completely under greedy evaluation, the reasons for which are discussed in section 7.

6.4 Policy Coverage

Policy coverage is measured as the fraction of the 14,400 discrete states whose Q-values were updated at least once during training:

$$\text{coverage} = \frac{|\{(s,a) \in Q : Q(s,a) \neq 0\}|}{14{,}400}. \tag{14}$$

Coverage varied across algorithms, but was overall consistent (around 35% coverage). Tabular REINFORCE showed the lowest effective coverage: while states may be visited, the per-state logit table receives sparse gradient signal due to infrequent revisits and the high-variance MC return. This directly explains the failure of the greedy policy, as states encountered during evaluation but rarely visited during training receive near-uniform action preferences.

7 Discussion

7.1 Why SARSA Outperformed Q-Learning

The performance gap between SARSA and Q-Learning can be attributed to the interaction between discretisation aliasing and the off-policy bootstrap target. Discretisation necessarily maps multiple continuous states to the same bin. Within a bin, some states may have a clearly optimal action while others do not. Q-Learning's $\max_a Q(s', a)$ operator selects the action with the highest Q-value in the next bin, which may correspond to a subset of the aliased continuous states. This leads to systematic overestimation of the value function in ambiguous bins, creating Q-values that look promising in aggregate but fail when the agent commits to them deterministically.

SARSA's on-policy bootstrap target $Q(S_{t+1}, A_{t+1})$ reflects the expected value under the agent's actual ε-greedy behaviour policy, which includes the possibility of sub-optimal

actions. This conservative estimate is more robust to aliasing because it accounts for the uncertainty of operating in a discretised approximation of a continuous space.

7.2 Implications for Credit Assignment

The Optuna search converged to $\lambda \approx 0.94$ for SARSA(λ). At this value, the eligibility trace for a state visited k steps ago carries weight 0.94^k, giving an effective credit horizon of approximately $\log(0.05)/\log(0.94) \approx 47$ steps. A short trace ($\lambda \ll 1$) would fail to assign credit to actions taken during the approach phase; a full Monte Carlo trace ($\lambda = 1$) would include the random early exploration phase and add unnecessary variance. The preference for $\lambda = 0.94$ therefore reveals that long but bounded credit assignment is beneficial.

This finding has a direct implication for the REINFORCE algorithm, suggesting a neural network implementation. That is, if the task benefits from propagating credit across near 50 steps, then a neural network that generalises across states and propagates gradients through the full episode is well-positioned to exploit this structure. without the variance penalty of full Monte Carlo, because function approximation reduces the effective variance through generalisation.

7.3 REINFORCE

The failure of the tabular REINFORCE implementation is not a consequence of a bug or a poor hyper-parameter choice. It reflects three compounding structural limitations of applying a Monte Carlo policy gradient method to a tabular representation of a large state space.

First, the variance of the Monte Carlo return is proportional to the episode length. With episodes of 200–500 steps, the return G_0 can range from -500 to -100, and its variance across episodes is large relative to the signal from any single state-action pair. Second, with 14,400 discrete states and episodes of ~ 300 steps, each episode visits approximately 2% of the state space. After 600 training episodes, most states are visited fewer than ten times, leaving the policy logits and baseline values poorly estimated. Third, the γ^t factor in the policy gradient update, which is theoretically correct per the pseudocode in Sutton and Barto, further attenuates the gradient for states encountered early in long episodes, precisely where strategic information about the swing-up approach is encoded. These three factors combine to produce a policy that never meaningfully differentiates actions in the majority of states, causing complete failure under greedy evaluation.

8 Conclusion

This report presented the implementation, optimisation, and evaluation of five reinforcement learning algorithms on the Acrobot-v1 environment: SARSA, Q-Learning, n-step SARSA, True Online SARSA(λ), and REINFORCE with baseline in tabular settings.

The principal experimental finding is that SARSA(λ) achieves the best greedy performance (-126.05 mean reward, 100% success rate), outperforming all single-step TD methods. The consistent outperformance of multi-step methods supports the hypothesis that Acrobot's swing-up structure benefits from credit assignment over a horizon of 20–50 steps.

Three methodological contributions emerged from the development process. First, ε-decay was reparametrised as a `floor_fraction` of the training budget, making the exploration schedule transferable between the HPO budget and the final training run. This fix prevented a failure mode that caused 0% greedy success in early runs of n-step SARSA and SARSA(λ). Second, the $\lambda = 0.94$ finding from HPO provides interpretable evidence that the task requires long-horizon credit assignment, a conclusion that is not easily obtainable from tabular TD(0) alone. Third, the principled failure of tabular REINFORCE, diagnosed as a combination of high MC variance, sparse state coverage, and attenuated γ^t gradients, directly motivates the neural network extension found in the appendix and illustrates the fundamental difference between value-based and policy gradient methods under tabular representation.

A main Limitations of this study is the sensitivity of all tabular methods to the choice of discretisation bins. Future work could extend the comparison to deep RL methods such as DQN, A2C, or PPO, which combine the function approximation benefits with more sample-efficient update schemes.

References

[1] Farama Foundation. *Gymnasium: A Standard API for Reinforcement Learning Environments*. 2023. URL: https://gymnasium.farama.org.

[2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.

[3] Takuya Akiba et al. "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *Proceedings of the 25th ACM SIGKDD*. 2019.

[4] R. Müller. *Hyperparameters in Reinforcement Learning and How To Tune Them*. TransferLab — appliedAI Institute. 2024. URL: https://transferlab.ai/pills/2024/hyperparameters-in-rl-and-how-to-tune-them/ (visited on 04/21/2026).